

C++

Programming:

An Object-Oriented Approach

Chapter 14

Exception Handling

C++

Programming:

An Object-Oriented
Approach



BEHROUZ A. FOROUZAN
RICHARD F. GILBERG

BEHROUZ A. FOROUZAN
RICHARD F. GILBERG

Objectives Part 1

To discuss traditional approaches to error handling.

To discuss exception handling in a function using three different patterns that use *try-catch* block and *throw* statements.

To discuss exception specifications to define what type of an exception a function can throw including any exception, pre-defined exceptions, and no exception.

To discuss the process of stack unwinding and its effect on catching exceptions.

Objectives Part 2

To discuss exceptions in classes and how exceptions should be handled in constructors using a version of try-catch block called function-try block.

To emphasize that we should avoid exceptions in destructors because it undermines the process of stack unwinding.

To discuss the general layout and format of standard exception classes in C++ giving their public interfaces and their purposes.

To show how we can create our own exception classes that are inherited from the standard exception classes.

INTRODUCTION PART 1

When we write source code, we expect that there could be some errors during the compilation.

Many new programmers, however, think that if a program compiles successfully, everything is fine.

Experienced programmers, on the other hand, are not happy until they test each program with a pre-determined set of test data.

INTRODUCTION PART 2

Even when a complete scheduled testing phase is used, an occasional error can occur when the program is run.

An error like this is referred to as an exception because it is rare.

The subject of this chapter is how to handle these exceptions.

INTRODUCTION PART 3

When we write source code, we expect that there could be some errors during the compilation.

Many new programmers, however, think that if a program compiles successfully, everything is fine.

Experienced programmers, on the other hand, are not happy until they test each program with a pre-determined set of test data.

INTRODUCTION PART 4

Even when a complete scheduled testing phase is used, an occasional error can occur when the program is run.

An error like this is referred to as an exception because it is rare.

The subject of this chapter is how to handle these exceptions.

Traditional Approaches to Error Handling

Part 1

Run-time errors are problematic.

If there are no syntactical errors, the program will be compiled, but a run-time error occurs that prevents the continuation of the program and it needs to be aborted.

There are several approaches that have been used in the past to deal with a run-time error.

We discuss four here.

Let Run-Time Environment Abort the Program

Probably the first approach is to do nothing and let the program terminate when there is an exception.

Traditional Approaches to Error Handling

Part 2

Program 14.1 *Dividing two integers*

```
1  /*****
2  * The program shows how division by zero aborts the program *
3  *****/
4  #include <iostream>
5  using namespace std;
6
7  int main ()
8  {
9      int num1, num2, result;
10     for (int i = 0; i < 5; i++)
11     {
12         cout << "Enter an integer: ";
13         cin >> num1;
14         cout << "Enter another integer: ";
15         cin >> num2;
16         result = num1 / num2; // The statement that may create exception
17         cout << "The result of division is: " << result << endl;
18     }
19     return 0;
20 }
```

Traditional Approaches to Error Handling

Part 3

Program 14.1 *Dividing two integers*

Run:

```
Enter an integer: 12
Enter another integer: 5
The result of division is: 2
Enter an integer: 10
Enter another integer: 3
The result of division is: 3
Enter an integer: 6
Enter another integer: 0
```

Traditional Approaches to Error Handling

Part 4

Ask Run-Time Environment to Abort the Program

A second approach that we have seen is to test for possible errors.

The advantage to this approach is that a message can be printed to explain what caused the program to abort.

Program that Forces Abortion in Division

Part 1

Program 14.2 Forcing abortion in division

```
1  /*****
2  * A program that aborts with an error message.          *
3  *****/
4  #include <iostream>
5  #include <cassert>
6  using namespace std;
7
8  int main ()
9  {
10     int num1, num2, result;
11     for (int i = 0; i < 5; i++)
12     {
13         cout << "Enter an integer: ";
14         cin >> num1;
15         cout << "Enter another integer: ";
16         cin >> num2;
17         if (num2 == 0)
18         {
19             cout << "No division by zero!. Program is aborted." << endl;
20             assert (false);
```

Program that Forces Abortion in Division

Part 2

Program 14.2 Forcing abortion in division

```
21         }
22         result = num1 / num2;
23         cout << "The result of division is: " << result << endl;
24     }
25     return 0;
26 }
```

Run:

```
Enter an integer: 8
Enter another integer: 3
The result of division is: 2
Enter an integer: 9
Enter another integer: 6
The result of division is: 1
Enter an integer: 7
Enter another integer: 0
No division by zero. Program is aborted.
```

Use Error Checking

Another approach is to check the value of the second number in each iteration and skip the division if it is zero.

This is preferred to the previous two methods because we can skip the cases in which the second number is zero and continue with the other cases.

Traditional Error Checking Program Part 1

Program 14.3 A traditional error-checking program

```
1  /*****
2   * A program that uses traditional error checking to prevent      *
3   * program abortion                                             *
4   *****/
5  #include <iostream>
6  using namespace std;
7
8  int main ( )
9  {
10     int num1, num2, result;
11     for (int i = 0; i < 4; i++)
12     {
13         cout << "Enter an integer: ";
14         cin >> num1;
15         cout << "Enter another integer: ";
16         cin >> num2;
17         if (num2 == 0)
18         {
19             cout << "Division cannot be done in this case." << endl;
20         }
```

Traditional Error Checking Program Part 2

Program 14.3 A traditional error-checking program

```
21     else
22     {
23         result = num1 / num2;
24         cout << "The result of division is: " << result << endl;
25     }
26 }
27 return 0;
28 }
```

Run:

```
Enter an integer: 8
Enter another integer: 2
The result of division is: 4
Enter an integer: 7
Enter another integer: 1
The result of division is: 7
Enter an integer: 8
Enter another integer: 0
Division cannot be done in this case.
```

Traditional Error Checking Program Part 3

Program 14.3 *A traditional error-checking program*

```
Enter an integer: 9  
Enter another integer: 7  
The result of division is: 1
```

Using Function Return Value for Error Checking 1

Execution of a statement in a C++ program always occurs in a function, not outside a function.

We may initialize variables outside a function (in the global area), but initialization is not execution; if there is a problem with the initialization, it is detected by the compiler.

Since execution always occurs in a function, we can say that the run-time error always occurs in a function.

Using Function Return Value for Error Checking 2

In procedural programming in the past, it was customary to perform each calculation in a function and the error checking was achieved by checking the return value of the function.

An example is the *main* function that returns 0 if there is no problem, and returns other values if there is a problem.

Programmers in the past used this idea and designed programs in which each function returned a specific value if there was an error.

Error Checking Using a Function Part 1

Program 14.4 Error checking using a function

```
1  /*****
2   * A program that uses the return value of a function to show *
3   * the occurrence of a run-time error                        *
4   *****/
5  #include <iostream>
6  using namespace std;
7
8  // Function declaration
9  int quotient (int first, int second);
10
11 int main ()
12 {
13     int num1, num2, result;
14     for (int i = 0; i < 3; i++)
15     {
16         cout << "Enter an integer: ";
17         cin >> num1;
18         cout << "Enter another integer: ";
19         cin >> num2;
20         result = quotient (num1, num2);
```

Error Checking Using a Function Part 2

Program 14.4 Error checking using a function

```
21     if (result == -1)
22     {
23         cout << "Error, division by zero." << endl;
24     }
25     else
26     {
27         cout << "Result of division is: " << result << endl;
28     }
29 }
30 return 0;
31 }
32
33 // Function definition
34 int quotient (int first, int second)
35 {
36     if (second == 0)
37     {
38         return -1;
39     }
40     return (first / second);
```

Error Checking Using a Function Part 3

Program 14.4 Error checking using a function

```
41 }
```

Run:

Enter an integer: 6

Enter another integer: 5

The result of division is: 1

Enter an integer: 7

Enter another integer: 0

Error, division by zero.

Enter an integer: 8

Enter another integer: 2

The result of division is: 4

Problems with Traditional Approaches

We now analyze these approaches to see why we need a new approach (exception handling).

- a. The first approach is the worst. We let the program abort without any warning.**
- b. The second approach is better, the program will still abort, but the user will be notified.**
- c. The third approach is better than the first two because the pair that would cause the program to abort is ignored, and the program continues with the rest of the data. The problem with this approach is that the code for handling an error is mingled with the productive code of the program.**
- d. The fourth approach is the best, but it cannot be applied in all cases. In addition, the principle of modular programming dictates that the return value of a function should be used only for one purpose, not two.**

Exception Handling Approach

To avoid the four problems mentioned in the previous section, C++ developed the exception handling approach.

In this approach, the run-time system detects the error, but it does not abort the program.

It lets the program handle the error.

In the exception handling approach, the run-time error is detected, but the program handles the error and aborts the program only if necessary.

Try-Catch Block Part 1

The exception-handling approach in C++ uses what is called the *try-catch* block.

This block is made of two clauses.

The first clause, which is called the *try* clause, includes the code that may cause the program to abort.

The run-time environment tries to execute the code.

If the code can be executed, the program flow continues.

If the code cannot be executed, the system *throws* an *exception*, but it does not abort the program.

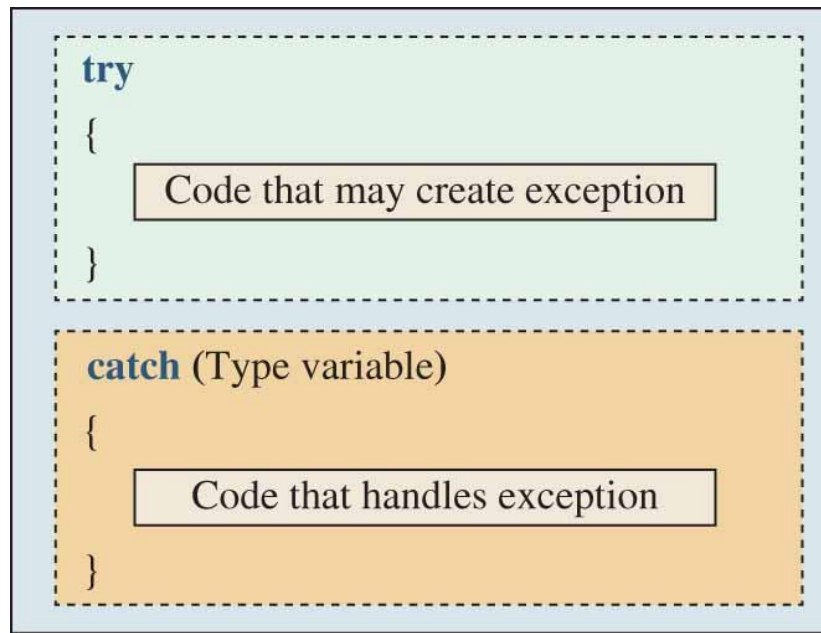
The second clause, called the *catch clause*, lets the program handle the exception and continue with the rest of program if possible.

Try-Catch Block Part 2

Figure 14.1 shows the simple *try-catch* block.

Later in the chapter we will see that *try-catch* blocks can have more than one *catch* clause.

Figure 14.1 *A simple try-catch block*



Notes:

The *try* clause detects the possibility of error and throws an exception object.

The *catch* clause handles the exception to prevent abortion.

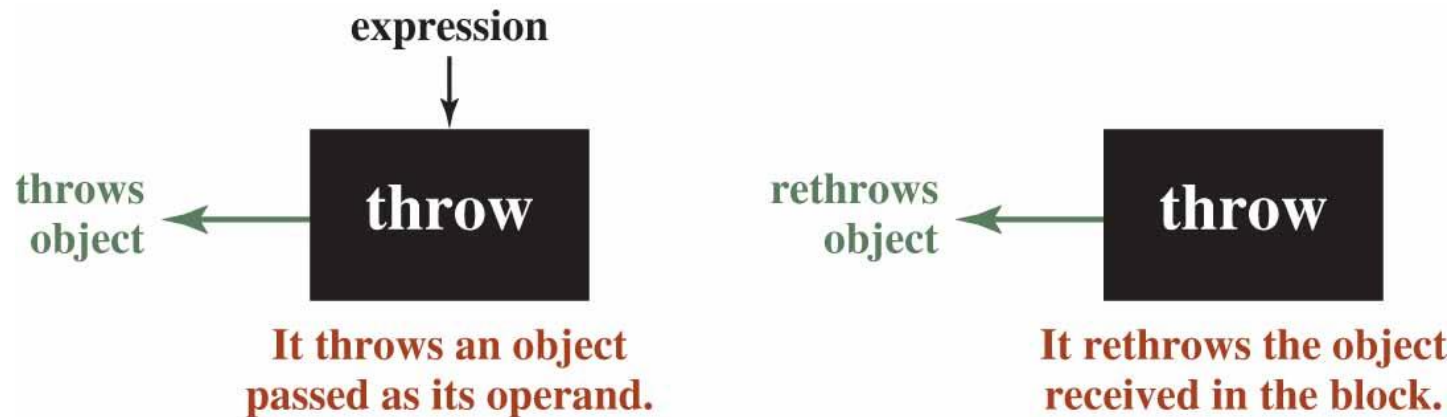
The two clauses must be one after the other without any code in between; they belong to the same block.

Using a Throw Expression to Throw an Object

Figure 14.2 shows the two versions of the *throw* operator.

The first is used to throw an exception; the second is used to rethrow an exception (we will discuss it later).

Figure 14.2 *Using a throw expression to throw an object*



Note that the expression has no return value. It is called only for its side effect, which is throwing an exception.

It is usually changed to an expression statement by adding a semicolon after the expression.

Three Patterns Part 1

We see three different patterns with exception handling approaches.

First Pattern

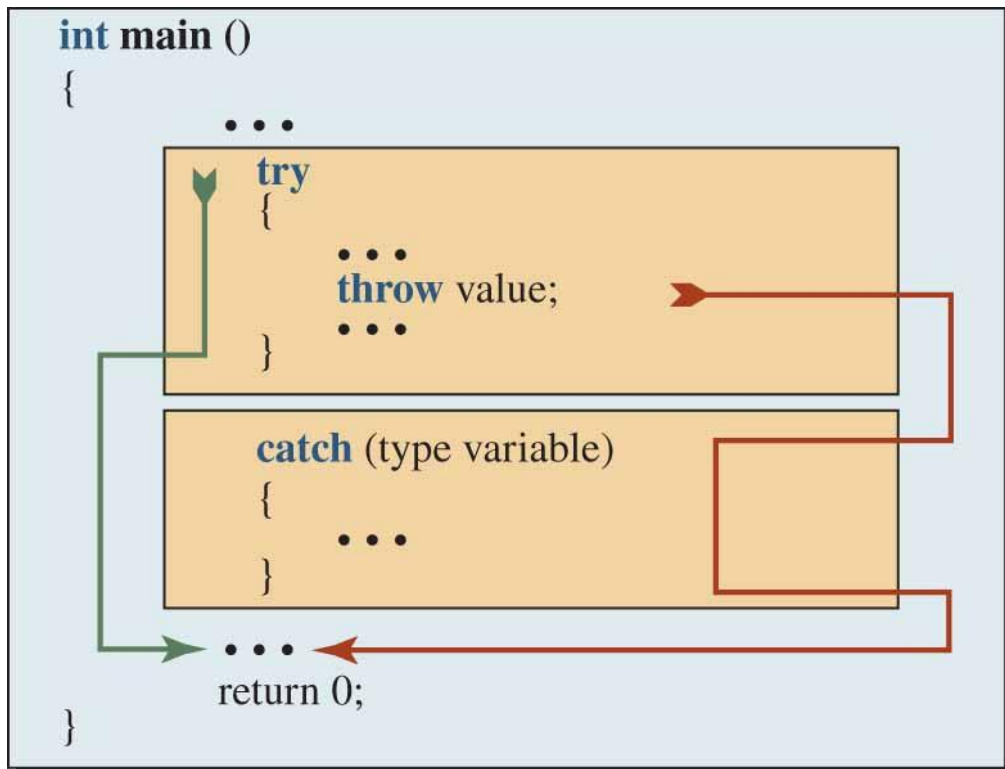
In the first pattern, the *try-catch* block is completely contained in one function.

If an exception is thrown, the rest of the try clause after the throw statement is ignored and control moves to the *catch* clause.

The function continues after the *catch* clause unless the program is aborted in the *catch* clause (Figure 14.3).

Three Patterns Part 2

Figure 14.3 *First pattern (try-catch block in the calling function)*



Legend:



Note:

The throw statement is directly enclosed in the *try* block.

Program Using Try-catch Block Part 1

Program 14.5 Using a try-catch block

```
1  /*****
2  * A program that uses the try-catch block to detect an error *
3  * and throw it to be caught and handled by the program      *
4  *****/
5  #include <iostream>
6  using namespace std;
7
8  int main ()
9  {
10     int num1, num2, result;
11     for (int i = 0; i < 3; i++)
12     {
13         cout << "Enter an integer: ";
14         cin >> num1;
15         cout << "Enter another integer: ";
16         cin >> num2;
17         // The try-catch block
18         try
19         {
20             if (num2 == 0)
```

Program Using Try-catch Block Part 2

Program 14.5 Using a try-catch block

```
21         {
22             throw 0; // An object of type integer is thrown
23         }
24         result = num1 / num2;
25         cout << "The result is: " << result << endl;
26     }
27     catch (int x)
28     {
29         cout << "Division by zero cannot be performed." << endl;
30     }
31 }
32 return 0;
33 }
```

Program Using Try-catch Block Part 3

Program 14.5 Using a try-catch block

Run:

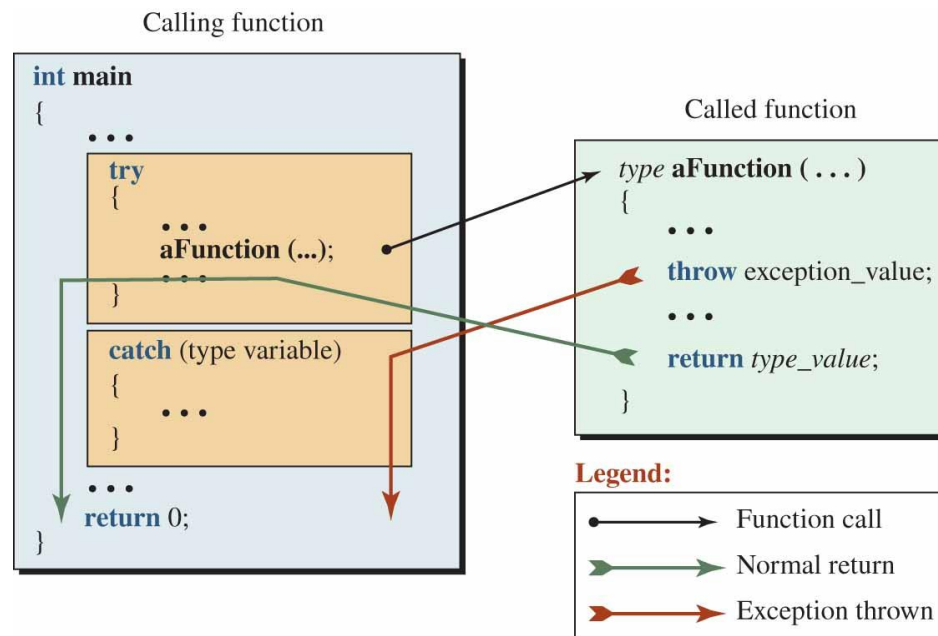
```
Enter an integer: 6
Enter another integer: 5
The result is: 1
Enter an integer: 7
Enter another integer: 0
Division by zero cannot be performed.
Enter an integer: 8
Enter another integer: 2
The result is: 4
```

Three Patterns Part 3

Second Pattern

In the second pattern, the *try-catch* block is still in the one function, but the exception is thrown in another function that is called in the *try* clause.

Figure 14.4 *Second pattern (throw in called function)*



Detecting an Exception Thrown by a Function Part 1

Program 14.6 Detecting an exception thrown by a function

```
1  /*****
2  * A program that uses the try-catch block to detect an error *
3  * thrown by a function                                     *
4  *****/
5  #include <iostream>
6  using namespace std;
7
8  int quotient (int first, int second); // Function declaration
9
10 int main ()
11 {
12     int num1, num2, result;
13     for (int i = 0; i < 3; i++)
14     {
15         cout << "Enter an integer: ";
16         cin >> num1;
17         cout << "Enter another integer: ";
18         cin >> num2;
19         // The try-catch block
20         try
```

Detecting an Exception Thrown by a Function Part 2

Program 14.6 Detecting an exception thrown by a function

```
21     {
22         cout << "Result: " << quotient (num1, num2) << endl;
23     }
24     catch (int ex)
25     {
26         cout << "Division by zero cannot be performed." << endl;
27     }
28 }
29 return 0;
30 }
31 // Function definition
32 int quotient (int first, int second)
33 {
34     if (second == 0)
35     {
36         throw 0;
37     }
38     return first / second;
39 }
```

Detecting an Exception Thrown by a Function Part 3

Program 14.6 *Detecting an exception thrown by a function*

Run:

Enter an integer: 12

Enter another integer: 4

Result: 3

Enter an integer: 16

Enter another integer: 0

Division by zero cannot be performed.

Enter an integer: 7

Enter another integer: 2

Result: 3

Detecting an Exception Thrown by a Function Part 4

The pattern allows a function to throw an exception to its caller.

In other words, the responsibility of throwing an exception and handling an exception is divided between the called function and the calling function.

We create a function, we do not need to reveal its code; we can just tell the users what it does (public interface) and tell them what possible exceptions it may throw.

The responsibility of the users is to catch and handle the exception. The benefit can be clearly seen in the case of library functions.

One of the advantages of the exception-handling approach is that we can design functions that can throw an exception. It is the responsibility of the caller to handle the exception.

Third Pattern Part 1

Sometimes we need to have a *try-catch* block in the called function.

This may occur when the called function belongs to an independent entity (such as a member function in a class).

When a function is called by another function, if an exception is caught and handled, we have two cases.

Third Pattern₁

- a.** In the first case, the called function can continue with the rest of its code and return to the calling function when it is finished.

This is the first pattern in which the called function acts independently with relation to the exception handling.

- a.** In the second case, the called function cannot continue with the rest of its code.

Third Pattern₂

- c. The control, however, needs to return to the calling function.**

Otherwise, it means aborting the program.

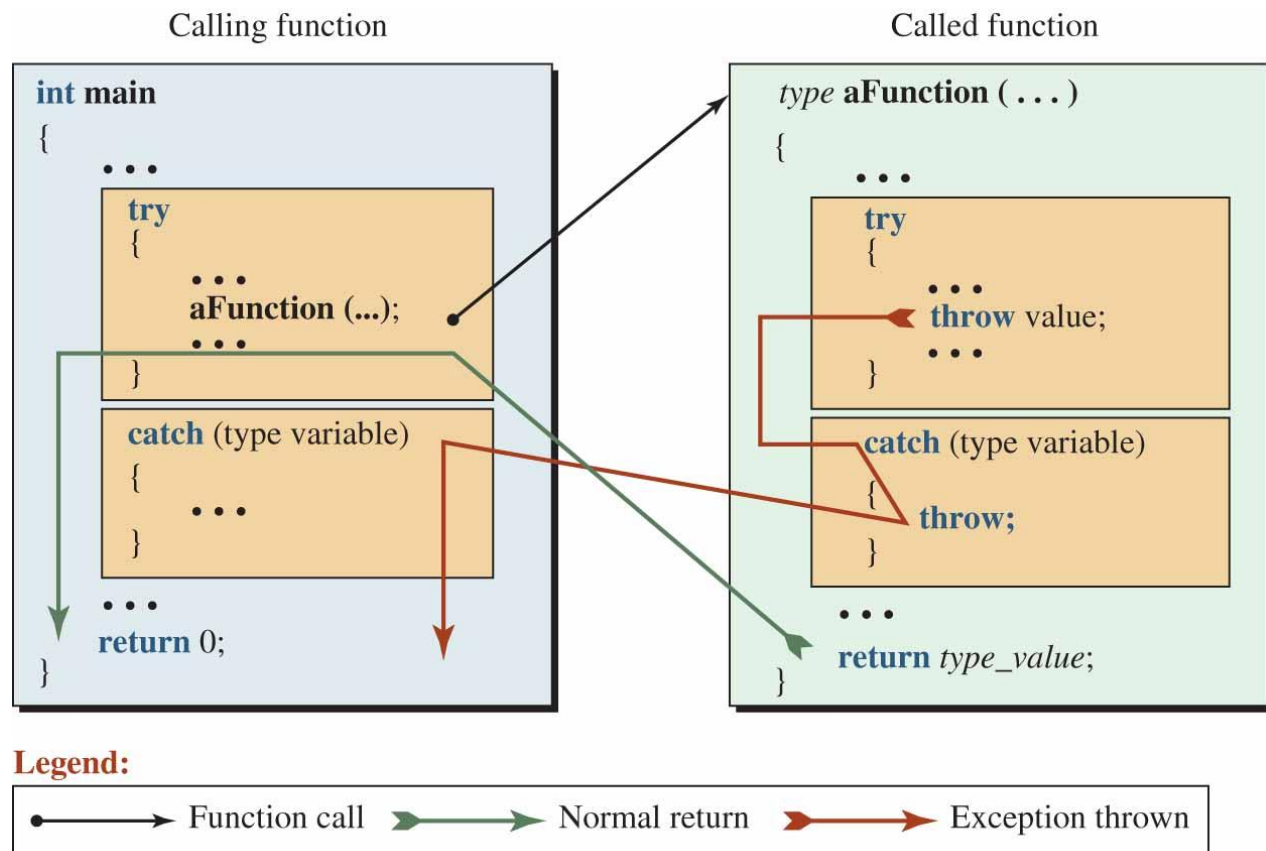
This is the third pattern.

We need to have the *try-catch* block in both functions.

The catch clause *rethrows* the exception to the calling function to be caught there. Figure 14.5 shows this pattern.

Try-catch Block in Calling and Called Function

Figure 14.5 *Third pattern (try-catch block in calling and called function)*



Note:

We have only one actual throw statement, which is enclosed in its own try block. The second throw statement simply rethrows the previous one.

Position of Throw Statement

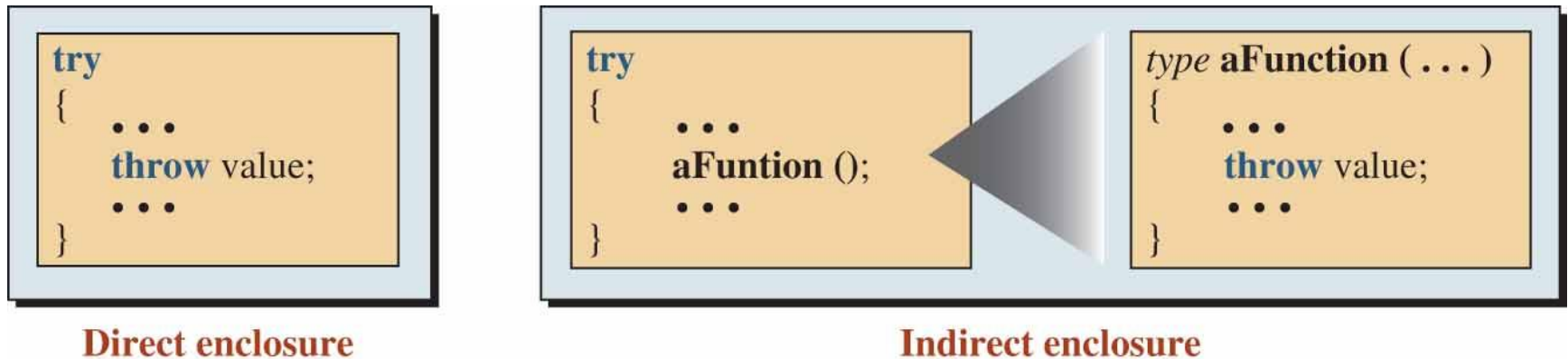
We need to make it clear that a throw statement needs to be enclosed in a try clause, either directly or indirectly, to catch the exception.

When the throw statement is explicitly inside a try clause, the enclosure is direct (first pattern); when it is inside a function that is called in a try clause, the enclosure is indirect (second pattern).

The third pattern we discussed is a special case in which there are two throw statements. Figure 14.6 shows the difference between the direct and indirect try clause.

Direct and Indirect Throw Statements

Figure 14.6 *Direct and indirect throw statements*



Hidden Throw Statement

Sometimes we see a *try-catch* block but we cannot see the *throw* statement.

This happens when we use a pre-defined or a library function whose definition we cannot see.

We just see the function call in a *try* clause, but the function definition contains a *throw* statement.

For example, some string member functions, such as the *at ()* function, throws an exception if the index is out of range.

When we call this function in our program, we may need to enclose the call in a *try-catch* block to catch the exception and prevent the abortion of the program.

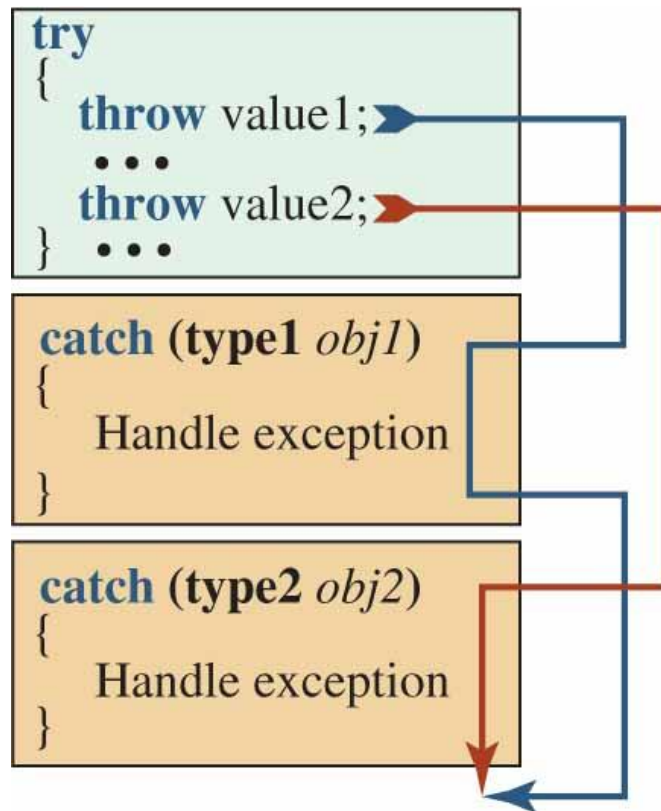
Multiple Catch Clauses

Figure 14.7 shows a situation in which a function may throw two different types of exceptions.

Since the handling for each type may be different, we can have two types of *catch* clauses.

Try-catch Block with Multiple Catch Clauses

Figure 14.7 *A try-catch block with multiple catch clauses*



Note:

If a value of **type1** is thrown, only the first catch clause is executed and the second is skipped. If a **type2** value is thrown, the first catch clause is skipped and the second catch clause is executed.

Legend:

→ value1 is thrown
→ value2 is thrown

Generic Catch Clause Part 1

A *catch* clause may only catch exceptions of its parameter type.

A *catch* clause of an integer type cannot catch an exception of a floating-point type when it is thrown.

If we want to catch any type of exception, we can use an ellipsis as the parameter in the *catch* clause.

In this case, if specific exception types need to be handled differently, the generic (any type) *catch* clause is coded as the last *catch* clause.

The following shows such as a *try-catch* block.

Generic Catch Clause Part 2

```
try
{
    ...
}
catch(int x)           // Specific type catch
{
    ...
}
catch(...)              // Ellipsis means any exception type
{
    ...
}
```

Exception Propagation₁

A function may throw an exception, but it does not means that the exception should be necessarily caught and handled in the same function where it has been thrown.

If an exception is not caught and handled where it has been thrown, it will be automatically propagated to the previous function in the hierarchy of function calls until one of the functions in the path captures and handles it.

Exception Propagation₂

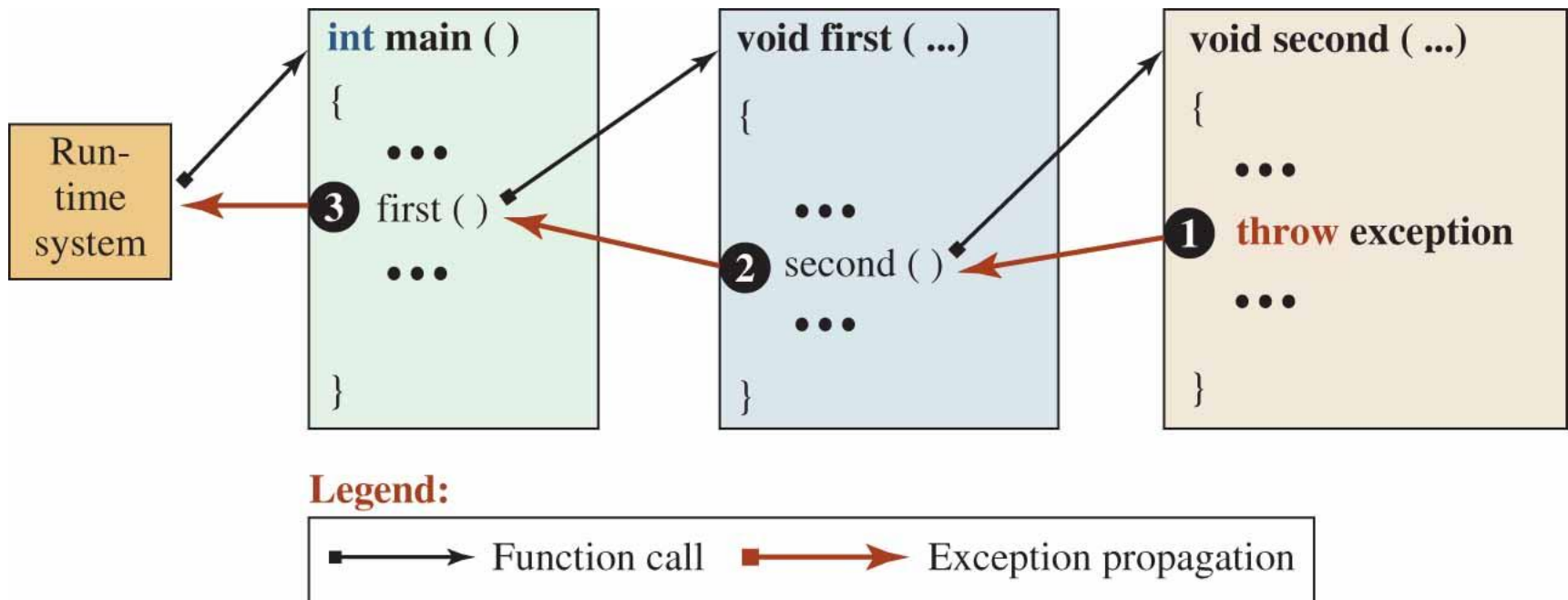
The last function in which a thrown exception can be caught and handled is the *main* function.

If it is not caught and handled by the *main* function it will be propagated to the run-time system, where it is caught and causes the *main* function to be aborted, which aborts the whole program.

Figure 14.8 shows the path of function calls and the path of exception propagation in three functions: *main* is called by run-time system, *first* is called by *main*, and *second* is called by *first*.

Exception Propagation Part 1

Figure 14.8 *Exception propagation*



Exception Propagation Part 2

An exception is thrown (explicitly or implicitly) in *second*, which is not caught.

The exception is propagated to *first*, where it is not caught. The exception is finally propagated to run-time system, where is caught and the program is terminated.

If any of the functions (*second*, *first*, or *main*) use the *try-catch* block and handle the propagation, it does not reached the run-time system and the program is not aborted.

Rethrowing An Exception

It is possible that a catch clause cannot or does not want to handle the exception either partially or totally.

In this case, the exception can be rethrown to the function one-level before.

This may happen when we need to free memory, close a network connection, or perform another maintenance activity before handling the exception.

```
try
{
    ...
}
catch(type variable)
{
    ...          // Some work
    throw;      // Re-throw the exception to the calling function
}
```

Exception Specification Part 1

When we write a function for our own use, we know what type of exception can be thrown in the function and we can create the appropriate *try-catch* block in the calling function.

However, if we write a function to be used by others, we normally give the user only the signature of the function as the public interface.

In other words, the user has no knowledge of the body of the function to find what exceptions can be thrown.

Exception Specification Part 2

In this case, it is recommended that an expression in the function header be added to tell the user that if she wants to catch an exception in the calling function, what will be the type of the thrown objects.

For this purpose, we can say that a function can be designed in one of the three ways: *any exception*, *pre-defined exceptions*, or *no exception*.

Any Exception Part 1

If there is no specification, the function looks like the ones we have written so far. This means that the function can throw any exception.

To find out what exception is thrown, we need to look at the body of the function.

This means that this case is not suitable for functions that are designed by one entity and used by another such as a library function.

The following shows the format of the prototype for this type of a function.

```
type functionName (parameters) ;    // Prototype with no specification
```

Any Exception Part 2

An example of this type of function is the one we used in Program 14.6.

The quotient function can throw any exception.

Since we have designed and used the function, we know that it throws an exception of type *int*.

```
int quotient(int first, int second);           // Function declaration
```

Pre-defined Exceptions

If the designer and the user of the function are different, we need to define the exceptions thrown by the function in the header of the function (which is copied in the declaration).

The syntax for this type of a function is shown below.

The header of the function defines the types of all exception objects that may be thrown from the function.

```
type functionName(parameters) throw(type1, type2, ..., typen);
```

We could have added a pre-defined specification to show that our quotient function may throw an exception of type *int* as shown below:

```
int quotient(int first, int second) throw(int); // declaration
```

No Exception

The third possibility is to declare to the user that this function does not throw an exception, which means that the user does not need to use a *try-catch* block.

The syntax for this type of a function is shown below. The header of the function uses the *throw* keyword, but the parentheses are empty.

```
type functionName(parameters) throw();
```

The following shows an example of this specification.

If a function is supposed to print two integers, there are no error conditions and no need for exception handling.

```
int print(int first, int second) throw(); // Function declaration
```

Stack Unwinding Part 1

One of the issues that we need to fully understand when handling exceptions is stack unwinding.

Stack unwinding is closely related to how the memory assigned to a program is managed.

Stack Unwinding Part 2

Program Memory

When a C++ program starts running, the run-time system designates four areas of memory for the program: *code memory*, *static memory*, *stack memory*, and *heap memory*.

The code memory (program memory) is that area of memory that holds the instructions of the program executed during the program run time.

Static memory stores the value of static variables and global variables.

Stack Unwinding Part 3

Program Memory

Stack memory is a last-in first-out memory similar to the stack of trays we see in a restaurant.

The last tray pushed into the stack is the first one to be popped out.

Stack memory is responsible for keeping track of three types of information about each function: values of parameters, values of local variables, and the return address of the calling function in code memory.

Heap memory is free memory that is used by a program to store information that may last more than the life of a function.

Stack Unwinding Part 4

Pushing and Popping Stack Memory

Exception handling is based on the behavior of stack memory.

Each program is made of a set of functions that may call each other.

During a function call, data about the calling function, such as parameters, local variables, and the return address, are pushed into stack memory.

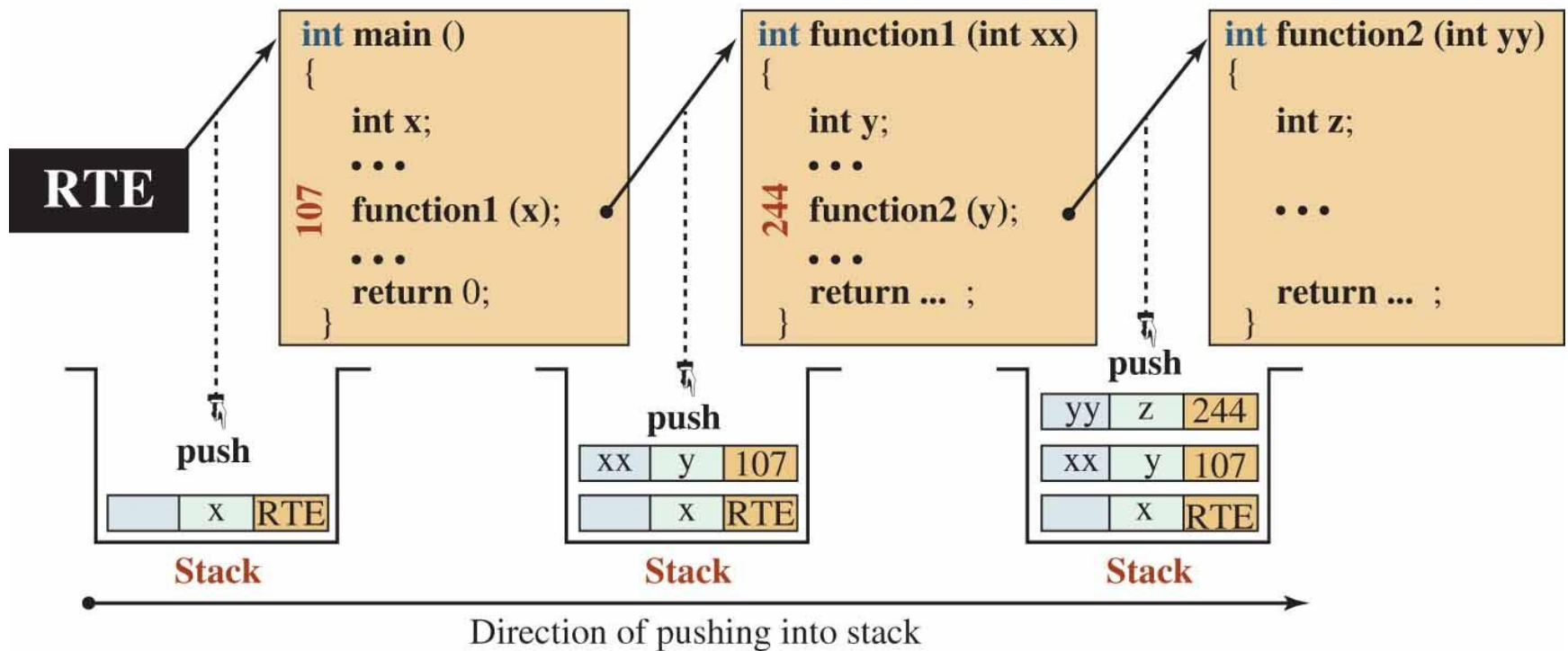
The return address is used when the function terminates normally or abnormally (Figure 14.9).

Pushing Function Information into the Stack

Figure 14.9 *Pushing function information into the stack*

RTE: Run-time environment

107 and 244 addresses in code memory



Pushing Function Information Into the Stack Part 1

In this example, the *main* function is pushed into the stack, there are no parameters, but the local variable (*x*), and the address of the instruction that called the *main* function in the RTE is pushed in the stack.

When the main function calls *function1*, the parameter of the *function1*(*xx*), the local variable (*y*), and the address of function call instruction (107) are pushed into the stack.

Similarly, when *function1* calls *function2*, the parameter of *function2* (*yy*), the local variable of *function2*(*z*), and address of function call (244) are stored in the stack.

Pushing Function Information Into the Stack Part 2

Now the stack has three entries because of three function calls.

Note that each parameter entry can have zero or more parameters and each local-variable record can have zero or more variables.

Figure 14.10 shows how the information is popped from the stack during the return from each function.

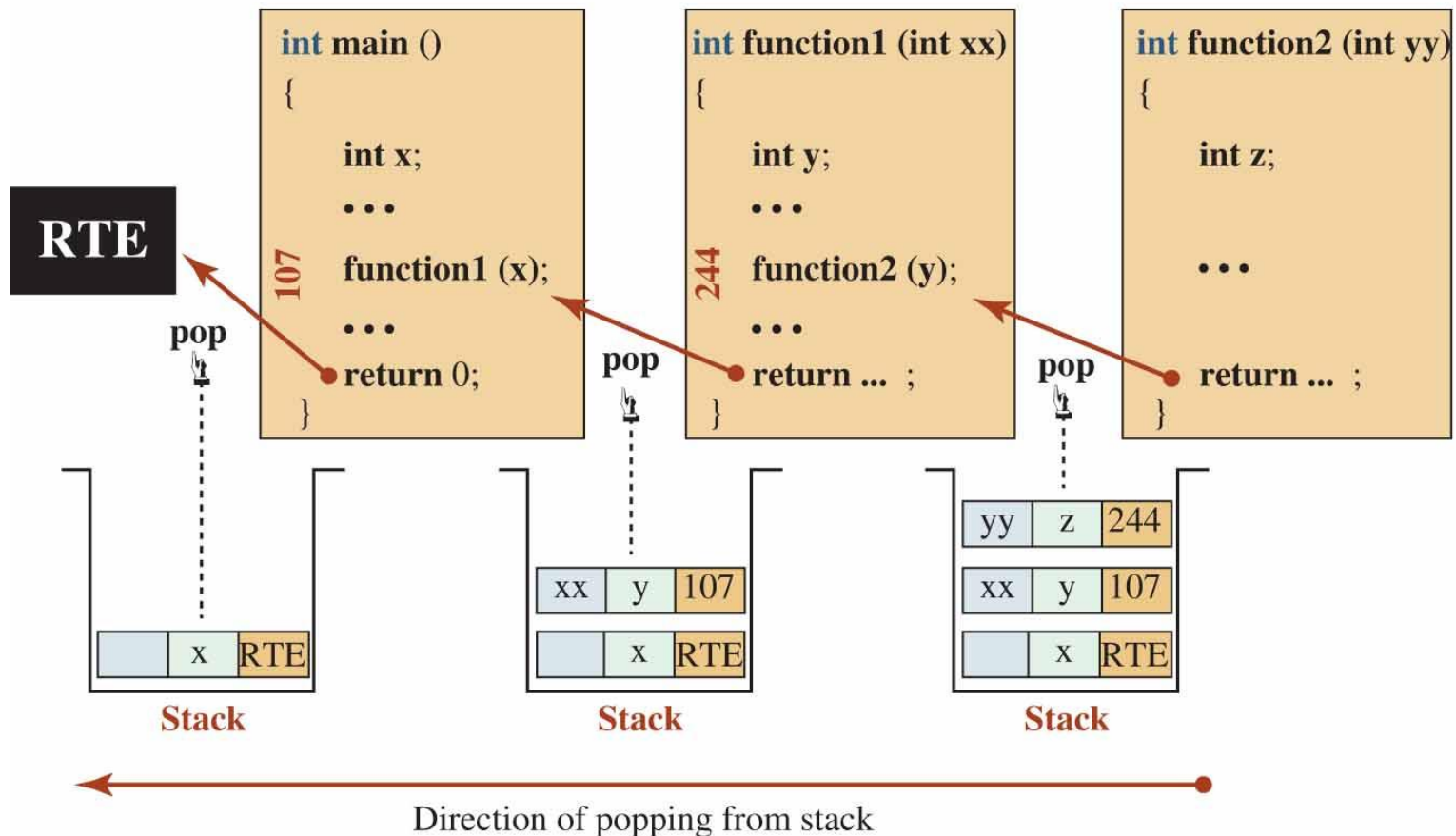
This is referred to as stack unwinding.

How Information Is Popped Up From the Stack₁

Figure 14.10 *How information is popped up from the stack*

RTE: Run-time environment

107 and 244 addresses in code memory



How Information Is Popped Up From the Stack₂

Popping of entries from stack memory due to a function return or a thrown exception is referred to as stack unwinding. During stack unwinding, the parameters and local objects of the function are automatically destroyed by calling their destructors.

To avoid memory leak, we need to make our objects local to the function in which they are defined.

EXCEPTIONS IN CLASSES

An exception may be thrown in any function defined in a class.

Although handling exceptions in member functions other than constructors and destructors is the same as the exceptions in a stand-alone function, we need to be careful about exceptions in constructors and destructors.

Exception in Constructors₁

A constructor is a function, but it is different from a regular function in that it is designed to create and initialize an object.

Before we learn how to catch an exception in a constructor, let us see what happens when an exception is thrown in a constructor.

We consider two cases.

In the first, the object is totally created in stack memory; in the second, it is partially created in heap memory.

Exception in Constructors₂

Creation of Objects in the Stack

To better understand the construction and destruction of an object when the object is totally created in stack memory, we consider two cases.

In the first, no exception is thrown in the constructor.

In the second, an exception is thrown in the constructor.

We assume that we have a class named Integer that stores one single integer.

This type of a class is referred to as a wrapper class and very common in object-oriented programming.

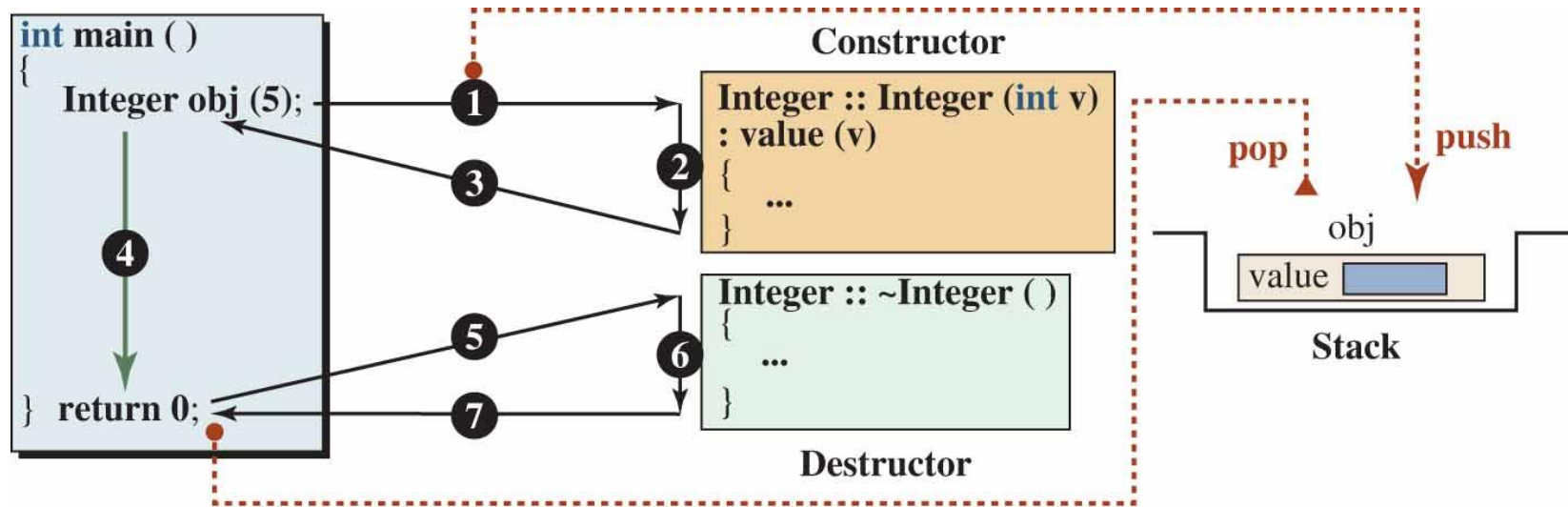
Exception in Constructors Part 1

Case 1: No Exception is Thrown in Constructor

When we are using stack memory for storing an object, the object is created on the top of the stack when the constructor is called and it is destroyed when the destructor completes.

Object Created in the Stack and No Exception

Figure 14.11 *Object created in the stack and no exception*



Notes:

1. An object is created and pushed into the stack when the constructor is called.
2. Execution of the constructor initializes the object and allocates resources, if any.
3. The initialized object is returned to the `main` function for use.
4. The `main` function uses the object.
5. When the object goes out of scope, the destructor is called.
6. The destructor deallocates the resources, if any.
7. The object is popped from the stack.

Exception in Constructors Part 2

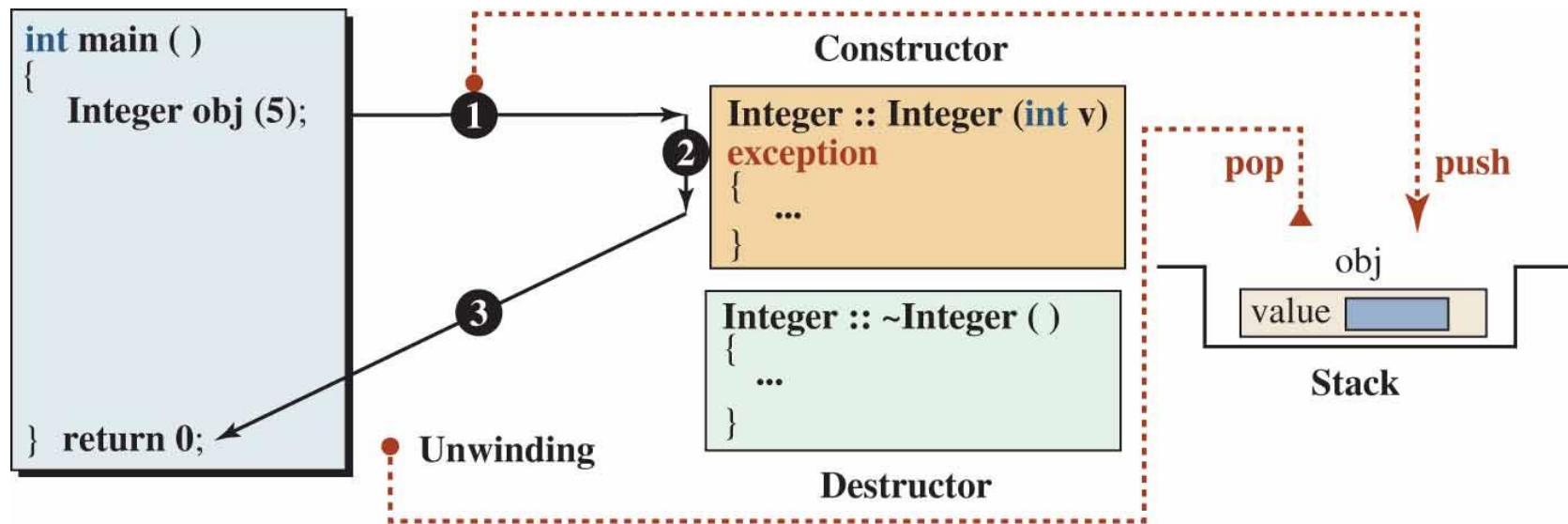
Case 2: An Exception is Thrown in the Constructor

C++ is designed such that if the constructor of a class cannot fully do its job, the destructor for that object is never called.

Figure 14.12 shows the same scenario as case 1, but the constructor throws an exception and the program terminates. All memory allocated in the stack is released.

Object Created in Stack and an Exception

Figure 14.12 *Object created in stack and an exception*



Notes:

1. An object is created and pushed into the stack when the constructor is called.
2. An exception occurs in the constructor during initialization (or allocating resources).
3. Throwing an exception terminates the constructor. The destructor is never called, but stack unwinding pops the object and it is destroyed at this point.

Exception in Constructors Part 3

Partial Creation of Object in the Heap

To better understand the construction and destruction of an object when the object is partially created in the heap, we assume that the Integer object has a pointer to an integer created in heap memory.

Again, we consider two cases.

Exception in Constructors Part 4

Case 1: No Exception is Thrown in Constructor

Figure 14.13 shows the case when no exception is thrown in the constructor.

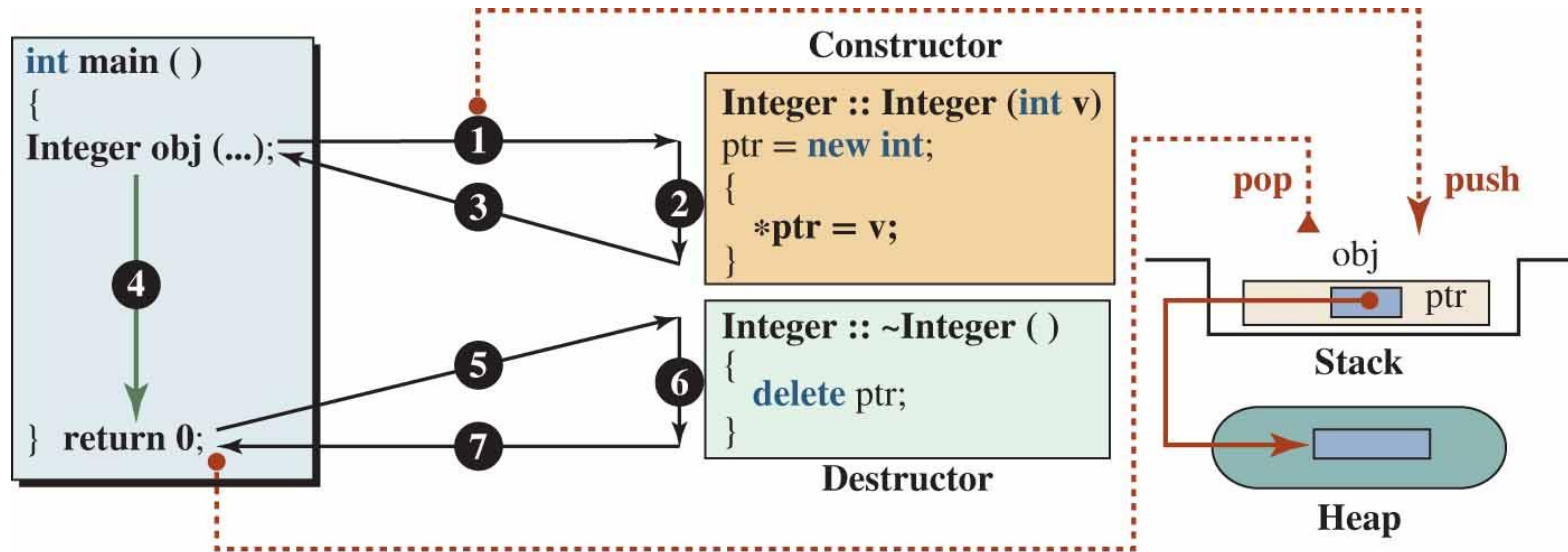
We assume that the constructor creates the object in the heap during the initialization and then stores a value in the object during the execution of the body.

When the *Integer* object goes out of scope, the destructor is called and it frees the memory.

After memory is freed, the *Integer* object is popped from stack memory.

Object Created in the Heap and No Exception

Figure 14.13 Object created in the heap and no exception



Notes:

1. An object is created and pushed in the stack when the constructor is called.
2. Initialization allocates memory in the heap and the value is stored in it during the execution of the constructor body.
3. The constructor is terminated and an object is available to the `main` function.
4. The `main` function uses the object.
5. When the object goes out of scope, the destructor is called.
6. The destructor deletes the integer in the heap.
7. The object goes out of scope after the destructor returns and the object is popped.

Exception in Constructors Part 5

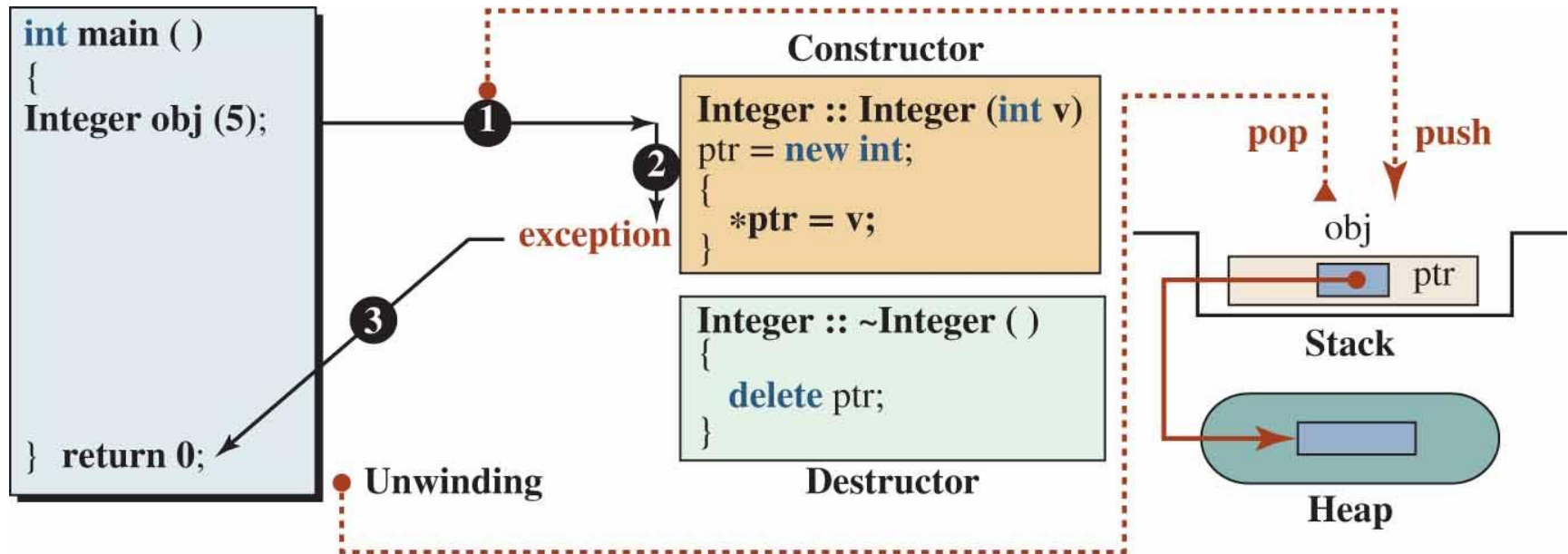
Case 2: An Exception is Thrown in Constructor

Since the object is not completely constructed, the destructor is not called, which means the memory allocated in the heap is never deleted.

We may have memory leak.

Object Created in the Heap and an Exception₁

Figure 14.14 Object created in the heap and an exception



Notes:

1. An object is created and pushed into the stack when the constructor is called.
2. Allocation of the location in the heap memory is done, but an exception occurs when the constructor needs to store values in the object allocated in the heap.
3. An exception is thrown that terminates the `main` function. Stack unwinding pops the `Integer` object from the stack, but the memory in the heap is not released because the destructor is not called.

Object Created in the Heap and an Exception₂

The problem here is that we separated the two related tasks of allocating memory and releasing memory by another task (storing data in the allocated memory).

Because this third task failed (an exception is thrown), the constructor has allocated memory, but its work is not finished.

This means that the destructor is not called to release the memory.

The two related tasks of allocating and releasing memory are partially done.

Memory is allocated, but not released. We may have memory leak.

Using a Smart Pointer for Memory Management 1

To link the allocation and release of memory together, we use a *smart* pointer instead of a raw pointer.

The *smart* pointer allocates memory when its constructor is called and releases memory when its destructor is called.

However, the constructor has no other duty.

Which means when allocation is over, the construction is complete and the destructor will be called by the unwinding if any exception is thrown.

Using a Smart Pointer for Memory Management 2

In other words, since the constructor of the *smart* pointer is doing only one job, its destructor is always called and we do not have any problem releasing memory.

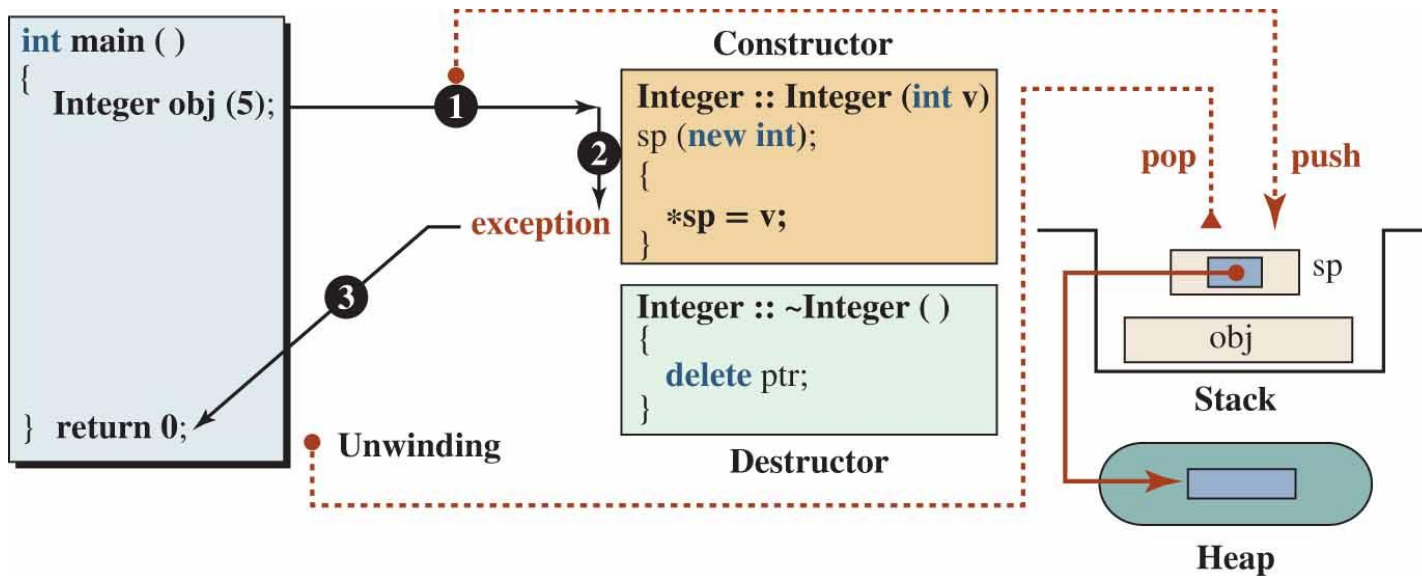
Figure 14.15 shows the same scenario but we are using a *smart* pointer instead of a raw pointer.

We may wonder what happens if the allocation of memory in the heap fails.

The answer is that there is no allocated memory that we should worry about.

Using a Smart Pointer

Figure 14.15 *Using a smart pointer*



Notes:

1. An *Integer* object is created in the stack, which in turn creates an SP object and pushes it into the stack.
2. In the constructor, the SP object creates a location in the heap, but during filling the location, an exception is thrown, which means the constructor of the *Integer* class has not totally constructed the object.
3. The *main* function terminates, and unwinding pops the SP object from the stack. Since the SP object was fully constructed, the destructor of the SP object is called, which deletes the memory on the heap. The destructor of the *Integer* object is not called, but we do not have a memory leak because the SP object released the memory.

Try-Catch Block in Constructor: Function-Try Block

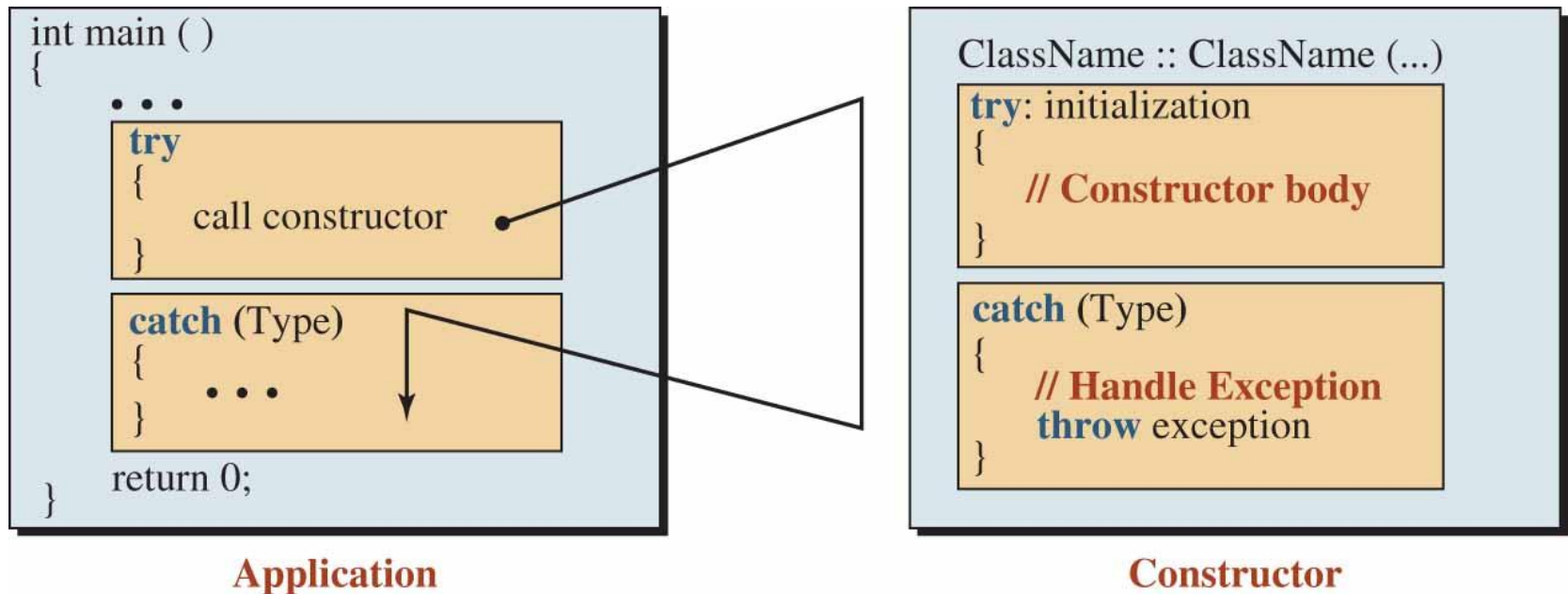
C++ does not allow us to throw, implicitly or explicitly, an exception in the initializer list.

To be able to throw an exception if it happens in the initializer list, we need to use what is called a *function-try* block, which combines a *try-catch* block with an initialization list as shown in Figure 14.16.

The *function-try* block allows the initialization to be added after the keyword *try* separated by the colon.

Use of the Function-try Block

Figure 14.16 *Use of the function-try block*



Interface File for Smart Pointer (SP) Class

Program 14.7 File *sp.h*

```
1  /*****
2  * The interface file for SP class
3  *****/
4  #ifndef SP_H
5  #define SP_H
6  #include <iostream>
7  using namespace std;
8
9  // Definition of class SP
10 class SP
11 {
12     private:
13         int* ptr;
14     public:
15         SP (int* ptr);
16         ~SP ( );
17         int& operator* ( ) const;
18         int* operator-> ( ) const;
19 };
20 #endif
```

Implementation File for SP Class Part 1

Program 14.8 File *sp.cpp*

```
1  /*****
2   * The implementation file for SP class *
3   *****/
4  #include "sp.h"
5
6  // Constructor
7  SP :: SP (int* p)
8  : ptr (p)
9  {
10 }
11 // Destructor
12 SP :: ~SP ()
13 {
14     delete ptr;
15 }
16 // Overloading of the * operator
17 int& SP :: operator* ( ) const
18 {
19     return *ptr;
20 }
```

Implementation File for SP Class Part 2

Program 14.8 File *sp.cpp*

```
21 // Overloading of the -> operator
22 int* SP :: operator-> ( ) const
23 {
24     return ptr;
25 }
```

Interface File for the Integer Class

Program 14.9 File *integer.h*

```
1  /*****
2   * The interface file for the Integer class          *
3   *****/
4  #ifndef INTEGER_H
5  #define INTEGER_H
6  #include "sp.h"
7
8  // Definition of the Integer class
9  class Integer
10 {
11     private:
12         SP sp;
13     public:
14         Integer (int value);
15         ~Integer ();
16         int getValue();
17 };
18 #endif
```

Implementation File for the Integer Class

Part 1

Program 14.10 File *integer.cpp*

```
1  /*****
2   * The implementation file for the Integer class          *
3   *****/
4  #include "integer.h"
5
6  // Constructor using function-try block
7  Integer :: Integer (int v)
8  try: sp (new int)
9  {
10     *sp = v;
11 }
12 catch (...)
13 {
14     throw;
15 }
16 // Destructor
17 Integer :: ~Integer ()
18 {
19 }
20 // Accessor function
```

Implementation File for the Integer Class

Part 2

Program 14.10 *File integer.cpp*

```
21 int Integer :: getValue()  
22 {  
23     return *sp;  
24 }
```

Application to Test the Integer Class Part 1

Program 14.11 File *app.cpp*

```
1  /*****
2  * The application to test the Integer class          *
3  *****/
4  #include "integer.h"
5
6  int main ( )
7  {
8      for (int i = 0; i < 1000000; i++)
9      {
10         try
11         {
12             Integer integer (i);
13             cout << integer.getValue() << endl;
14         }
15         catch (...)
16         {
17             cout << "Exception is thrown" << endl;
18         }
19     }
20     return 0;
```

Application to Test the Integer Class Part 2

Program 14.11 File *app.cpp*

```
21 }
```

Run:

```
0
1
2
3
4
5
6
...
87245
Exception is thrown
87247
...
92101
Exception is thrown
92103
...
99999
```

Exception in Destructors

We mentioned that destructors are invoked during stack unwinding.

If the destructor is interrupted by throwing an exception, it means that the unwinding process is stopped.

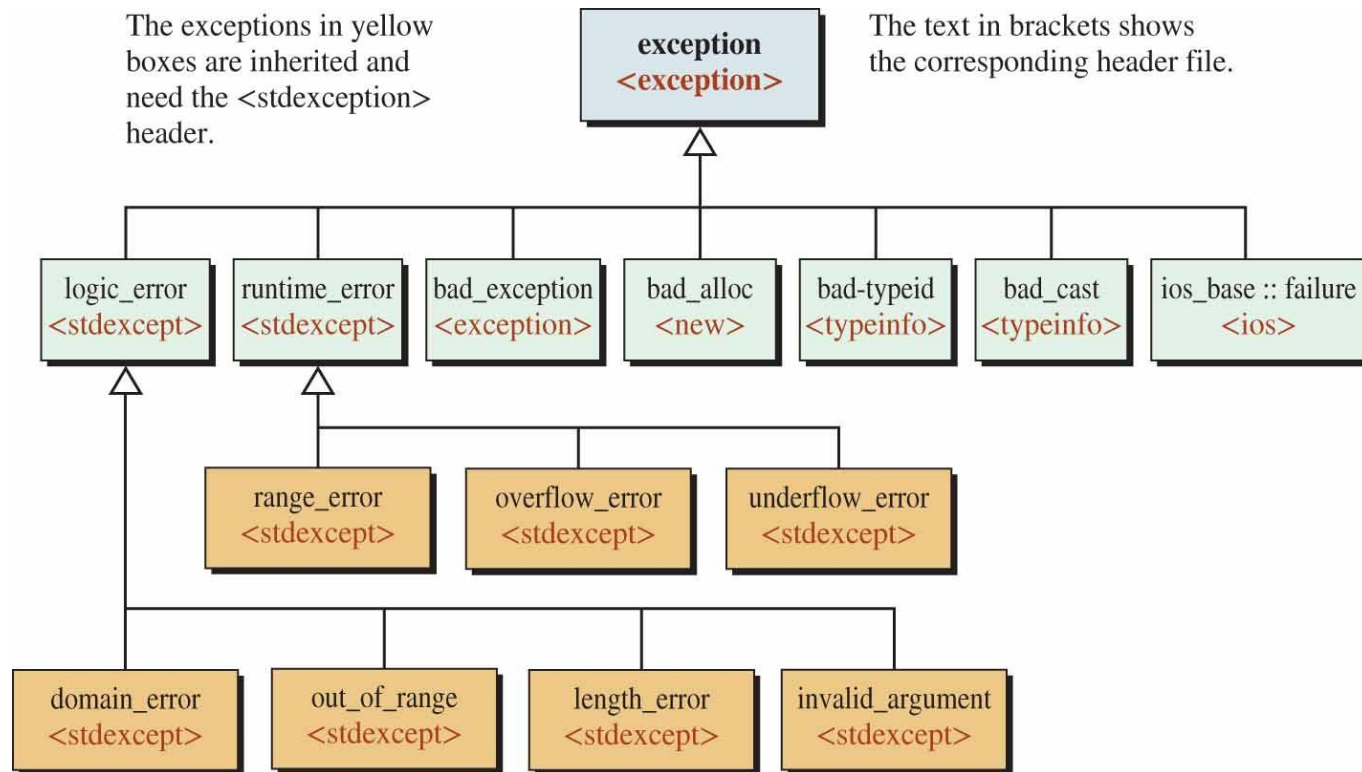
For this reason, if any exception is thrown in the destructor, C++ calls a global function named *terminator* that terminates the whole program.

Exception throwing in a destructor must be avoided.

STANDARD EXCEPTION CLASSES PART 1

C++ defines a set of standard exception classes that are used in its library as shown in Figure 14.17.

Figure 14.17 *Hierarchy of standard exception classes*



STANDARD EXCEPTION CLASSES PART 2

At the top of the hierarchy is the *exception* class defined in the <exception> header.

Table 14.1 shows the public interface for the *exception* class.

Table 14.1 *Public interface of the exception class*

```
exception() throw() // constructor
exception(const exception&) throw() // copy constructor
exception& operator =(const exception&) throw() // Assignment operator
virtual ~exception() throw() // destructor
virtual const char* what() const throw() // member function
```

Logical Errors Part 1

As shown in Figure 14.17, C++ defines a class named *logic_error*. This is the base class for four other classes related to logic errors, errors that are related to the precondition for a function that cannot be detected during compilation time.

The logical errors are related to the preconditions of a function.

Table 14.2 shows the public interface for the *logical_error* class. These classes also inherit the *what* member function from the *exception* class.

Table 14.2 *Public interface for the logic_error class*

```
explicit logic_error(const string& whatArg) // constructor
virtual const char* what() const throw() // member function
```

Logical Errors Part 2

There are four classes derived from this class: *domain_error*, *length_error*, *out_of_range*, and *invalid_argument*.

The constructors of all of these classes have the same pattern as the *logic_error* class, but the names are different.

Table 14.3 shows the constructor of the four exception classes inherited from the *logic_error* class.

Table 14.3 *Classes inherited from logic_error class*

class	constructor
<code>domain_error</code>	<code>explicit domain_error(const string& whatArg)</code>
<code>length_error</code>	<code>explicit length_error(const string& whatArg)</code>
<code>out_of_range</code>	<code>explicit out_of_range(const string& whatArg)</code>
<code>invalid_argument</code>	<code>explicit invalid_argument(const string& whatArg)</code>

To use logical error classes, we need the `<stdexcept>` header file.

Logical Errors Part 3

The domain_error Class

The *domain_error* exception is thrown when the given data is out of domain.

For example, if a function requires an argument between 0.0 and 4.0 then this exception is thrown if we pass a value that is not in this domain.

The length_error Class

The *length_error* exception is thrown if the length of an object is greater or smaller than the predefined length.

For example, the string class throws an exception if the size of a string exceeds the value returned by the *max_size* member function.

Logical Errors Part 4

The out_of_range Class

The *out_of_range* exception is thrown if an index goes out of range for a library class.

For example, the string class has a function named *at(...)* that returns a value at the index defined as its parameter.

If an integer is passed that is beyond the index of the current string object, an exception of this type is thrown.

We can use this class to throw an exception if an the index of an array is out of range.

Logical Errors Part 5

The `invalid_argument` Class

The *`invalid_argument`* exception normally occurs when there is a logical error, but the nature of an exception does not match any of the previously-defined three classes.

An example is when we have a bit set in which each bit should have a value of either 0 or 1.

Run-Time Errors Part 1

C++ defines a class named *runtime_error* class. A run-time error is normally related to the post condition of a function such as overflow, underflow, or an out-of-range return value.

To use the runtime error classes, we need the `<stdexcept>` header file.

Table 14.4 shows that the public interface for the *runtime_error* class also inherits the *what* member function from the exception class.

Table 14.4 *Public interface for the runtime_error class*

```
explicit runtime_error(const string& whatArg)    // constructor
virtual const char* what() const throw()        // member function
```

Run-Time Errors Part 2

Three classes are derived from this class: *underflow_error*, *overflow_error*, and *range_error* (see Figure 14.17).

The constructors of all of these classes have the same pattern as the *runtime_error* class, but the names are different. The *what* function is inherited in all classes.

The run-time errors are related to the post-conditions of a function.

Table 14.5 *Classes inherited from runtime_error classes*

class	constructor
<i>underflow_error</i>	<code>explicit underflow_error(const string& whatArg)</code>
<i>overflow_error</i>	<code>explicit overflow_error(const string& whatArg)</code>
<i>range_error</i>	<code>explicit range_error(const string& whatArg)</code>

Run-Time Errors Part 3

The underflow_error Class

The *underflow_error* condition occurs in arithmetic calculations.

However, this type of error is not normally defined for any arithmetic operator. It can be used to throw an exception in a user-defined function.

The overflow_error Class

The *overflow_error* condition also occurs in arithmetic calculations.

However, this type of error is not normally defined for any arithmetic operator.

It can be used to throw an exception in a user-defined function.

Five Other Classes Part 1

The `bad_exception` Class

A *`bad_exception`* class object is thrown from a function with an exception specification that defines the function will not throw exceptions, but something happens in the function and it throws one.

The `bad_alloc` Class

The `<new>` header defines types and functions related to dynamic memory allocation.

The operator *`new`* throws an object of type *`bad_alloc`* class if the requested memory cannot be allocated.

Five Other Classes Part 2

The `bad_typeid` Class

The *`bad_typeid`* exception is thrown when we define a type which cannot be fulfilled.

For example, if we try to dereference a pointer (*P) which is null, an exception of this type will be thrown.

The `bad_cast` Class

The *`bad_cast`* class is used to throws an exception when a dynamic cast operation fails.

Five Other Classes Part 3

The failure Class

The `<ios>` header defines a class that can be used as the base class for exceptions thrown in all input/output classes.

Note that the name of the class is *failure* but its scope is *ios_base*.

The only parameter in the constructor is used to customized the message to be shown by the *at()* function (Table 14.6).

Table 14.6 *The interface for the failure class*

<code>explicit failure(const string& mesg)</code>	<code>// constructor</code>
<code>virtual ~failure()</code>	<code>// destructor</code>
<code>virtual const char* what() const throw()</code>	<code>// member function</code>

Five Other Classes Part 4

The input/output classes in C++ were defined before the language supported exception handling.

However, after the exception class was added to the language, the class `failure` was added to the hierarchy of exception classes.

The input/output classes hold states that can show if any error has occurred in the corresponding stream.

To be able to use exceptions in input/output operation, C++ added two functions named *exception()* as shown in Table 14.7.

Table 14.7 *Interface for Input/output exception classes*

```
void exception(iostate flags) // set flags that triggered exception
iostate exception() const    // Returns flags candidate for exception
```

Using Standard Exception Classes

Instead of creating our own exception objects, we can use an object of one of the standard exception classes defined in the `<exception>` or `<stdexcept>` headers for this purpose.

Use of invalid_argument Class Part 1

Program 14.12 Use of invalid_argument class

```
1  /*****
2   * This program shows how to use an object of invalid_argument *
3   * to detect division by zero in a function.                      *
4   *****/
5  #include <stdexcept>
6  #include <iostream>
7  using namespace std;
8
9  // Function declaration
10 int quotient (int first, int second);
11
12 int main ()
13 {
14     int num1, num2, result;
15     for (int i = 0; i < 3; i++)
16     {
17         cout << "Enter an integer: ";
18         cin >> num1;
19         cout << "Enter another integer: ";
20         cin >> num2;
```

Use of invalid_argument Class Part 2

Program 14.12 Use of invalid_argument class

```
21      // Try-Catch block
22      try
23      {
24          cout << "Result of division: " << quotient (num1, num2);
25          cout << endl;
26      }
27      catch (invalid_argument ex)
28      {
29          cout << ex.what () << endl;
30      }
31  }
32  return 0;
33 }
34 // Function definition
35 int quotient (int first, int second)
36 {
37     if (second == 0)
38     {
39         throw invalid_argument ("Error! Divide by zero!");
40     }
```

Use of invalid_argument Class Part 3

Program 14.12 *Use of invalid_argument class*

```
41     return first / second;  
42 }
```

Run:

Enter an integer: 20

Enter another integer: 4

Result of division: 5

Enter an integer: 12

Enter another integer: 0

Error! Divide by zero!

Enter an integer: 14

Enter another integer: 5

Result of division: 2

Inheriting from Standard Exceptional Classes₁

Sometimes, none of the standard exception classes fit our need.

In this case, we can create an exception class that inherits from an appropriate exception class.

Assume we want to create a triangle class that creates triangle objects and finds their areas and perimeters.

We know that we need to take care of two separate errors.

First the measure of each side must be positive. Second, the following relationships must be satisfied.

```
side1 + side2 > side3  
side2 + side3 > side1  
side3 + side1 > side2
```

Inheriting from Standard Exceptional Classes₂

We can create our own exception class that handles both error conditions.

Program 14.13 shows the interface file for a customized exception class named `TriangleError` that inherits from the standard *`invalid_argument`* class.

Note that the constructor can accept an integer value that defines one of the error conditions.

Interface File for TriangleError Class

Program 14.13 The interface file for TriangleError class

```
1  /*****
2  * The interface file for a customized exception class      *
3  * inherited from invalid_argument class                    *
4  *****/
5  #ifndef TRIANGLEERROR_H
6  #define TRIANGLEERROR_H
7  #include <stdexcept>
8  #include <iostream>
9  #include <string>
10 using namespace std;
11
12 class TriangleError : public invalid_argument
13 {
14     private:
15         int errorType;
16     public:
17         TriangleError(int errorType) ;
18         string message() ;
19 };
20 #endif
```

Implementation File for TriangleError Class Part 1

Program 14.14 The implementation file for TriangleError class

```
1  /*****
2  * The implementation file a customized exception class      *
3  * inherited from invalid_argument class                    *
4  *****/
5  #include "triangleError.h"
6
7  // Definition of Constructor
8  TriangleError :: TriangleError(int err)
9  :errorType(err), invalid_argument("")
10 {
11 }
12 // Definition of message function
13 string TriangleError :: message()
14 {
15     if (errorType == 1)
16     {
17         return "Error!. One of the sides is negative! \n";
18     }
19     else
20     {
```

Implementation File for TriangleError

Class Part 2

Program 14.14 *Continued*

```
21     return "Error. The given sides do not define a triangle. \n";  
22     }  
23 }
```

Interface File for the Triangle Class Part 1

Program 14.15 The interface file for the Triangle class

```
1  /*****
2  * The interface file to define the Triangle class      *
3  *****/
4  #ifndef TRIANGLE_H
5  #define TRIANGLE_H
6  #include <cmath>
7  #include <iomanip>
8  #include "triangleError.h"
9
10 // Definition of Triangle class
11 class Triangle
12 {
13     private:
14         double side1;
15         double side2;
16         double side3;
17     public:
18         Triangle(double side1, double side2, double side3);
19         double getArea() const;
20         double getPerimeter() const;
```

Interface File for the Triangle Class Part 2

Program 14.15 *The interface file for the Triangle class*

```
21 };  
22 #endif
```

Implementation File for the Triangle Class

Part 1

Program 14.16 *The implementation file for the Triangle class*

```
1  /*****
2  * The implementation file for the Triangle class          *
3  *****/
4  #include "triangle.h"
5  #include "triangleError.h"
6
7  // Constructor with try block
8  Triangle :: Triangle(double sd1, double sd2, double sd3)
9  try: side1(sd1), side2(sd2), side3(sd3)
10 {
11     bool cond1 = (side1 > 0.0 && side2 > 0.0 && side3 > 0.0)
12     bool cond2 = side1 + side2 > side3;
13     bool cond3 = side2 + side3 > side1;
14     bool cond4 = side3 + side1 > side2;
15     if (! cond1)
16     {
17         throw TriangleError(1);
18     }
19     else if (!(cond2 && cond3 && cond4))
20     {
```

Implementation File for the Triangle Class

Part 2

Program 14.16 *The implementation file for the Triangle class*

```
21         throw TriangleError(2) ;
22     }
23 }
24 catch(...)
25 {
26     throw;
27 }
28 // The definition for getArea function
29 double Triangle :: getArea() const
30 {
31     double s = (side1 + side2 + side3) / 2;
32     return (sqrt(s * (s - side1) * (s - side2) * (s - side3)));
33 }
34 // The definition for getPerimeter function
35 double Triangle :: getPerimeter() const
36 {
37     return (side1 + side2 + side3);
38 }
```

Application File to Test the Triangle Class

Part 1

Program 14.17 *The application file to test the Triangle class*

```
1  /*****
2  * The application file to test the Triangle class      *
3  *****/
4  #include "triangle.h"
5
6  int main()
7  {
8      double side1, side2, side3;
9      for (int i = 0; i < 4; i++)
10     {
11         cout << "Enter the sides of triangle separated by space: ";
12         cin >> side1;
13         cin >> side2;
14         cin >> side3;
15         // The try-catch block
16         try
17         {
18             Triangle triangle(side1, side2, side3);
19             cout << "The area is: " << fixed << setprecision(2);
20             cout << triangle.getArea() << endl;
```

Application File to Test the Triangle Class

Part 2

Program 14.17 *The application file to test the Triangle class*

```
21         cout << "The perimeter is: " << fixed << setprecision(2);  
22         cout << triangle.getPerimeter() << endl;  
23     }  
24     catch(TriangleError te)  
25     {  
26         cout << te.message();  
27     }  
28 }  
29 return 0;  
30 }
```

Application File to Test the Triangle Class

Part 3

Program 14.17 *The application file to test the Triangle class*

Run:

Enter the sides of triangle separated by space: 3 4 5

The area is: 6.00

The perimeter is: 12.00

Enter the sides of triangle separated by space: 12 4 6

Error. The given sides do not define a triangle.

Enter the sides of triangle separated by space: 4 -5 7

Error!. One of the sides is negative!

Enter the sides of triangle separated by space: 8 5 4

The area is: 8.18

The perimeter is: 17.00

Memory Exception Class Part 1

Example

The following shows how we can inherit from the `bad_alloc` class to create a customized exception class named `MemoryException`.

```
class MemoryException: public bad_alloc
{
    public:
        MemoryException();
        const char* what() const throw();
};
```

```
MemoryException::MemoryException()
: bad_alloc()
{
}
const char* MemoryException::what() const throw()
{
    return "A memory failure has occurred.";
}
```

Memory Exception Class Part 2

Example

The following shows how to derive from the `out_of_range` exception class to throw an exception if an array index is out of range.

```
class ArrayIndexException : public out_of_range
{
    public:
        ArrayIndexException(const string& message);
};
```

```
ArrayIndexException::ArrayIndexException(const string& message)
: out_of_range(message)
{
}
const char* ArrayIndexException::what() const throw()
{
    return "Array index out of range";
}
```